

A MINI PROJECT REPORT

ON

"Medical Store Management System"

Submitted by

Priya

University Roll No. 2400950100066

B.Tech – Computer Science & Engineering, 5th Semester (3rd Year)

In partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MAHATMA GANDHI MISSION'S COLLEGE OF ENGINEERING & TECHNOLOGY, NOIDA

Affiliated to Dr. A.P.J. Abdul Kalam Technical University (AKTU), Lucknow

2026

CERTIFICATE

This is to certify that the project report entitled "Medical Store Management System" is a bona fide academic work carried out by Priya, University Roll No. 2400950100066, a student of Bachelor of Technology in Computer Science and Engineering, 5th Semester, Mahatma Gandhi Mission's College of Engineering & Technology, Noida.

The work has been prepared as an independent academic project during the academic year 2026-27. To the best of the knowledge of the undersigned, the project has not been submitted elsewhere for the award of any degree or diploma.

Project Guide / Faculty Coordinator:

Head of Department:

Date: 9 August 2026

DECLARATION

I, Priya, University Roll No. 2400950100066, hereby declare that the project report entitled "Medical StoreManagement System" is my original academic work. The report has been prepared for the Bachelor of Technology programme in Computer Science and Engineering at Mahatma Gandhi Mission's College of Engineering & Technology, Noida.

I further declare that the material presented in this report has not been copied from any other report or submitted to any other institution for the award of a degree. Wherever ideas, documentation, or published material have been consulted, suitable acknowledgement has been included in the references section.

Place: Noida

Date: 9 August 2026

Signature of Student:

Priya

ACKNOWLEDGEMENT

I express my sincere gratitude to the faculty members of the Department of Computer Science and Engineering at Mahatma Gandhi Mission's College of Engineering & Technology, Noida, for their guidance and encouragement during this project.

I am thankful to the Head of Department and the institute for providing the academic environment and resources needed to complete this work. Building the Medical StoreManagement System end-to-end required applying core concepts from software engineering, database management systems, and web technologies. I also acknowledge the official documentation of Node.js, Express, libSQL/SQLite, and Netlify that supported the design, implementation, and deployment of this project.

Finally, I thank my family and friends for their support and motivation throughout the development and documentation of this project.

Priya

University Roll No. 2400950100066

ABSTRACT

MediCare Pro is a web-based application developed to simplify day-to-day pharmacy and medical store operations. Keeping track of medicines, patients, and refill/return due dates by hand does not scale much beyond a few dozen products. A small pharmacy quickly needs a system that can tell, at a glance, which medicines are in stock, which patient currently holds a dispensed item, and which returns are overdue. This project replaces error-prone manual registers with a structured digital workflow.

The application uses HTML, CSS, and vanilla JavaScript for the user interface. Node.js and Express provide REST API endpoints, while a libSQL database (a SQLite-compatible engine) stores medicines, patients, and dispense records. The system follows a clear client-server design. The frontend is deliberately kept as a single dynamic page that swaps between a Dashboard, Medicines, Patients, and Dispense/Restock section without a page reload, communicating with the backend purely through fetch() calls to the JSON API. The backend is deployed as a serverless function on Netlify, with the same Express application wrapped for that environment, so the whole project runs as a single, easily hostable webapp.

Core pharmacy workflows are implemented with practical business rules rather than bare CRUD operations: a medicine can only be dispensed if stock is actually available, every dispensed medicine is due for refill/return in fourteen days, restocking automatically restores one available unit, and a late return is charged a surcharge of Rs. 5 per day past the due date. The application validates basic input, tracks available stock, and prevents the dispensing of out-of-stock medicines.

The final system demonstrates how a small full-stack web application can be designed using beginner friendly technologies while maintaining clear separation between the user interface, server routes, and database. It provides a highly responsive dashboard endpoint that aggregates total medicines, total patients, medicines currently dispensed, and overdue returns.

Keywords: Pharmacy automation, Medical store management, Node.js, Express, libSQL, SQLite, REST API, JavaScript, CRUD, Netlify, Web Application.

TABLE OF CONTENTS

Certificate	i
Declaration	ii
Acknowledgement.....	iii
Abstract.....	iv
Table of Contents	v
List of Abbreviations.....	vi
List of Tables.....	vii
1. Introduction.....	1
1.1 Project Background.....	1
1.2 Project Objective	2
1.3 Need for the System	3
2. Problem Statement and Objectives	4
2.1 Problem Statement.....	4
2.2 Scope of the Project.....	5
3. Requirement Analysis.....	6
3.1 Functional Requirements.....	6
3.2 Non-Functional Requirements	7
4. Feasibility and Methodology	8
5. Technology Stack.....	10
6. System Architecture	13
7. Database Design.....	15
8. Module Description.....	17
9. Workflow and Data Integrity.....	19
10. User Interface Design	21
11. Testing and Validation.....	23
12. Conclusion and Future Scope.....	25
References.....	27

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DBMS	Database Management System
GTIN	Global Trade Item Number (Barcode / Batch Code)
HTML	Hyper Text Markup Language
HTTP	Hyper Text Transfer Protocol
JS	JavaScript
JSON	JavaScript Object Notation
npm	Node Package Manager
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
UX	User Experience

LIST OF TABLES

Table No.	Title	Page
Table 3.1	Functional Requirements Breakdown	6
Table 3.2	Non-Functional Requirements Summary	7
Table 5.1	Core Technology Stack	10
Table 5.2	Deployment Configuration	12
Table 7.1	Database Tables and Relationships	15
Table 7.2	medicines (books) Table Field Design	16
Table 7.3	patients (members) Table Field Design	16
Table 7.4	dispense_records (issues) Table Field Design	16
Table 8.1	REST API Endpoints	18
Table 11.1	Test Cases and Expected Outcomes	24

1 INTRODUCTION

1.1 Project Background

Retail pharmacies and small medical stores handle a large, constantly moving inventory of medicines alongside a growing base of regular patients and walk-in customers. When these records are kept manually in registers, routine tasks such as checking whether a medicine is in stock, calculating a refill or return-due date, and identifying overdue returns of dispensed items take considerable time and are highly susceptible to human error. A small digital system can dramatically streamline these tasks, making them faster, more consistent, and far easier to audit.

MediCare Pro is a single-page web application designed to digitize and automate the day-to-day operations of a pharmacy or medical store. It manages medicine inventory, patient/customer records, and the dispense-and-restock cycle. Store staff can add and update medicine listings, search the inventory by name or manufacturer, dispense available stock to a registered patient, process restocks/returns, and view a live operational dashboard.

Technically, MediCare Pro was built by applying a dedicated Domain Translation Layer on top of an existing, general-purpose circulation-tracking application. The underlying relational schema, REST API contracts, and business-rule engine were kept fully intact, while every user-facing label, sample data set, and visual theme was re-mapped to pharmacy terminology — a catalog record became a medicine record, a registered user became a patient/customer, and a checkout/return transaction became a dispense/restock transaction. This report doubles as a case study in re-skinning a well-structured CRUD application for a different business domain without touching its data layer.

1.2 Project Objective

The primary objective of this project is to design, implement, and demonstrate a complete, easily deployable pharmacy / medical store management application that handles the everyday operations of a small store correctly, using a schema-preserving domain-translation approach. Specific technical objectives include:

- Reusing a well-normalized relational database consisting of three core tables: medicines, patients, and dispense records.
- Building a robust REST API using Node.js and Express that supports full CRUD operations for inventory, patients, and dispensing.
- Implementing core pharmacy business rules: enforcing a 14-day refill/return window, automatic stock-availability tracking, and automated late-return surcharge calculation.
- Ensuring data integrity by linking dispense records to valid medicines and patients using foreign keys.
- Building a single-page, no-framework frontend (vanilla HTML/CSS/JS) themed for a medical store, that dynamically renders data from the API.
- Providing an administrative dashboard that aggregates totals for medicines, patients, active dispenses, and overdue returns in real time.
- Demonstrating a reusable domain-translation methodology — proving the same schema and API can power more than one branded product simply by changing the presentation layer.
- Packaging the API as a serverless function so the application can be deployed on Netlify without managing a dedicated server.

1.3 Need for the System

The project demonstrates a practical use of database-driven software in a real-world retail/healthcare-adjacent scenario. Traditional manual pharmacy registers require staff to repeatedly calculate refill/return dates and manually cross-check stock availability against dispense logs. This system reduces duplicate entries, keeps stock levels perfectly synchronized with dispense records through database transactions, and gives store staff a quick, accurate overview of current inventory, active dispenses, and overdue returns at the click of a button.

2 PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

A manual or spreadsheet-based approach to running a small pharmacy or medical store breaks down in several predictable ways once the inventory or patient base begins to grow. The core problems identified include:

- **No Live Stock Count:** Manual registers do not automatically show the number of units available. Staff must physically check shelves or manually calculate based on dispense logs.
- **Overdue / Late-Return Tracking Errors:** Determining which dispensed items are overdue for return and how much surcharge has accumulated requires manual date arithmetic, which is tedious and error-prone.
- **Data Redundancy and Inconsistency:** Without a centralized database, patient details or medicine details might be duplicated across different physical registers.
- **No Consolidated Overview:** It is extremely difficult for store administrators to get a real-time summary of total dispensed items versus available inventory without spending hours tallying records.

The problem is to build a simple, efficient digital system that stores these records centrally, applies the core rules of pharmacy stock circulation automatically, and eliminates the need for manual record-keeping.

2.2 Scope of the Project

The current scope of MediCare Pro is defined as a web application intended for demonstration purposes or for deployment in a small pharmacy or medical store. It focuses on the most essential administrative operations.

The system includes:

- Complete management of the medicine inventory (adding, editing, deleting, searching).
- Complete management of patient / customer records.
- A transaction module for dispensing and restocking medicines.
- Automated late-fee calculation and refill-due-date assignment.
- A summary dashboard.

Items explicitly outside the current scope include online payment gateways, barcode/QR-code hardware scanning, automated email/SMS reminders, multi-branch store synchronization, prescription-image upload and verification, and role-based access control (RBAC) with multiple login tiers. These have been designated as future enhancements.

3 REQUIREMENT ANALYSIS

3.1 Functional Requirements

Functional requirements define the specific behaviours and functions the system must support. MediCare Pro encompasses four main functional domains:

Module	Core Functionalities
Medicine Inventory Management	Create, view, edit, delete, and search medicine information including name, manufacturer/brand, barcode/batch code, drug category, total stock, and available stock.
Patient / Customer Management	Create, view, edit, and delete patient information including name, email, phone, and registration date.
Dispensing (Dispense / Restock)	Dispense a medicine to a patient, set dispensed and refill-due dates automatically, block dispensing when no stock remains, process a restock/return, and calculate late surcharges.
Dashboard	Display real-time totals for medicines, patients, active dispenses, and overdue returns.

The interface is designed as a Single Page Application (SPA) so that store staff can smoothly switch between the Dashboard, Medicines, Patients, and Dispense/Restock sections without opening separate windows or triggering a full page reload.

3.2 Non-Functional Requirements

Non-functional requirements specify the quality attributes, performance goals, and constraints of the system. For this project, they are as follows:

Attribute	Requirement Description
Usability	Forms, buttons, tables, and feedback messages must be intuitive and clear to a first-time pharmacy staff member without extensive training.
Performance	Records should load quickly via asynchronous fetch operations for a small store environment.
Reliability	Database transactions must be used where stock availability and dispense records change together, to prevent partial updates.
Maintainability	Source code files must be modularly separated by concern (routes, database, static files) and contain clear plain-language comments.
Data Integrity	Foreign keys must strictly link dispense records to valid medicines and patients. Unique constraints on barcode/batch code and email must prevent duplicates.

4 FEASIBILITY AND METHODOLOGY

4.1 Technical Feasibility

The selected technology stack is highly technically feasible for the scope of this project. Node.js natively supports asynchronous JavaScript execution on the server, which is ideal for handling concurrent API requests from store terminals. The Express framework offers lightweight, minimal route handlers that map perfectly to RESTful design patterns for inventory, patients, and dispensing.

For data persistence, the project uses the `@libsql/client` driver, which speaks the SQLite wire protocol and can work against a local database file or a hosted libSQL database with identical application code. This keeps the querying model simple and SQL-based while remaining compatible with serverless hosting, where a plain local SQLite file is not always practical. The frontend strictly requires only a modern web browser and uses standard HTML5, CSS3, and vanilla JavaScript without the heavy dependencies of frameworks like React or Angular — which also made the pharmacy-domain re-theming straightforward, since presentation logic lives entirely in plain markup, CSS, and `script.js`.

4.2 Economic and Operational Feasibility

Economically, the system incurs zero licensing costs. All primary technologies (Node.js, Express, libSQL/SQLite, HTML/CSS/JS) are open-source and free for educational and commercial use, and the project can be hosted on Netlify's free tier using serverless functions, avoiding the cost of a dedicated always-on server.

Operationally, the digital workflow maps extremely closely to traditional, common pharmacy tasks (cataloguing medicines, registering patients, dispensing and restocking items). Therefore, pharmacy staff or store administrators can understand and operate the system with minimal training, ensuring high operational feasibility.

4.3 Development Methodology

An incremental, iterative approach was adopted for the software development lifecycle:

- Phase 1: Planning and Schema Reuse. The existing data tables and API route structures were reviewed to confirm they generalize cleanly to a medicine / patient / dispense-record domain without any schema changes.
- Phase 2: Domain Translation Mapping. Every database column, API field, and UI label was mapped from its original meaning to its pharmacy-domain equivalent (see Chapter 7 for the full mapping table).
- Phase 3: Backend Verification. The unchanged CRUD routes for medicines and patients were re-tested against the new sample data to confirm zero breaking changes.
- Phase 4: Circulation Logic Verification. The dispense/restock workflow — batched database writes, date calculations, and surcharge logic — was re-validated under the new terminology.
- Phase 5: Frontend Re-theming. The HTML/CSS layout was restructured with a teal-and-cyan medical design system, medical iconography, and pharmacy copywriting, while the JavaScript `fetch()` wiring to the backend was left untouched.
- Phase 6: Deployment. The Express application remained wrapped as a Netlify serverless function, with the same redirect rules serving the API and the re-themed single-page frontend from one domain.

- Phase 7: Refinement. Forms validation, CSS styling, responsive layout tweaks, and error messaging were refined based on manual testing of the pharmacy-themed UI.

5 TECHNOLOGY STACK

The project employs a lightweight, highly efficient stack designed to minimize overhead while showcasing how a domain-translation layer can sit cleanly on top of core web standards and a raw REST API, without introducing large frontend frameworks.

5.1 Frontend Technologies

The presentation layer runs entirely within the client's web browser.

- **HTML5:** Provides the semantic page structure, housing the navigation, forms, and inventory/patient/dispense tables.
- **CSS3:** Delivers a medical design system built around a teal/emerald primary palette (#0d9488, #0f766e, #14b8a6, #f0fdfa), a cyan accent (#0284c7, #06b6d4, #e0f2fe), and a dark-slate sidebar (#0f172a, #1e293b), using Flexbox and CSS Grid so the interface adapts from desktop monitors down to a mobile top bar and bottom navigation bar.
- **Vanilla JavaScript:** Updates the DOM dynamically. The native `fetch()` API sends asynchronous HTTP requests to the backend and injects the returned JSON data directly into the HTML tables — stock pills, drug-category chips, and status badges — without requiring a page reload.

5.2 Backend Technologies

The application logic layer handles routing, business rules, and API responses — carried over unchanged from the original system.

- **Node.js:** The JavaScript runtime environment that executes the server-side code.
- **Express.js:** A minimal and flexible web application framework that defines the RESTful endpoints for medicines, patients, dispense records, and the dashboard. It utilizes CORS and JSON middleware to parse incoming request payloads.
- **serverless-http:** A small adapter library that wraps the existing Express application so it can run inside a Netlify Function without rewriting any route logic.

5.3 Database Technologies

The data persistence layer ensures durable storage of pharmacy records.

- **@libsql/client:** The database driver used by the project. It speaks the same SQL dialect as SQLite and exposes a promise-based `execute()` method for single statements and a `batch()` method that runs multiple statements as one atomic write, used when dispensing or restocking a medicine.
- **Database File:** The physical database file was renamed from `library.db` to `medical.db` to match the new domain, while the connection module (`db.js`) and its internal table schemas were left unchanged.
- **Schema Initialization:** Tables are created automatically when the server starts if they do not already exist, and sample pharmaceutical inventory (e.g., Paracetamol 500mg, Amoxicillin 250mg, Ibuprofen 400mg, Lipitor 20mg, Omeprazole 20mg, Cetirizine 10mg) and sample patient records (e.g., Aarav Sharma, Diya Patel, Kabir Singh, Meera Nair) are seeded into an empty database for immediate testing.

5.4 Deployment Architecture

The application is configured for deployment on Netlify rather than a traditional always-on server. `netlify.toml` declares `public/` as the publish directory and `netlify/functions` as the functions directory, and defines two redirect rules: any request to `/api/*` is routed to the packaged Netlify Function, and any other request falls back to `index.html` so the single-page, pharmacy-themed frontend can handle client-side navigation. The function itself (`netlify/functions/api.js`) imports the same Express app used for local development and wraps it with `serverless-http`, so the exact same route handlers run in both environments.

Layer	Technology Used	Primary Function
Frontend UI	HTML5, CSS3, Vanilla JS	Presentation, User Input, Async Fetch
Backend API	Node.js, Express.js	Routing, Business Logic, Validation
Database	@libsql/client (SQLite-compatible)	Data Persistence, Integrity, Batched Writes
Deployment	Netlify Functions, serverless-http	Hosting the API and static frontend together

6 SYSTEM ARCHITECTURE

6.1 Architectural Design

The system follows a classic three-tier architectural design pattern. This promotes separation of concerns, making the codebase easier to read, maintain, and re-theme for a new domain.

- **Presentation Layer (Client):** The browser interface consisting of `index.html`, `style.css`, and `script.js` inside the public folder — carrying the MediCare Pro branding, medical iconography, and pharmacy copy. It is responsible only for rendering data and capturing user input.
- **Application Layer (Server):** The Express application in `server.js`, together with its route modules. It acts as the middleman, validating incoming HTTP requests, applying dispensing business rules (like calculating refill-due dates), and returning HTTP status codes with JSON payloads. In production, this same application is invoked through a Netlify Function instead of a long-running Node process.
- **Data Layer (Database):** The `libSQL/SQLite` database, accessed through `db.js`, which safely stores relational data and enforces strict constraints such as unique barcodes/batch codes and foreign-key relationships.

Data Flow: Browser (HTML/CSS/JavaScript) → HTTP Request → Netlify Redirect Rule → Express REST API (local process or Netlify Function) → SQL Query → `libSQL/SQLite` Database → JSON Response → Browser UI Update.

6.2 REST API Endpoints

The communication between the frontend and backend strictly follows REST principles. As part of the domain translation, the primary route paths were renamed to match pharmacy terminology — `/api/medicines`, `/api/patients`, and `/api/dispense` — while the legacy paths (`/api/books`, `/api/members`, `/api/issues`) remain mounted on the server as fallback aliases, so any existing integration built against the original naming continues to work without modification. The `/api/dashboard` path itself is unchanged.

Method	Endpoint Path	Action Description
GET	<code>/api/dashboard</code>	Returns aggregate counts: <code>totalMedicines</code> , <code>totalPatients</code> , <code>totalDispensed</code> , <code>totalOverdueRefills</code> .
GET	<code>/api/medicines</code>	Retrieves all medicines, accepts optional <code>?search</code> query.
POST	<code>/api/medicines</code>	Creates a new medicine product record in the inventory.
PUT	<code>/api/medicines/:id</code>	Updates an existing medicine product's details.
DELETE	<code>/api/medicines/:id</code>	Deletes a medicine product (only if no active or past dispense history).
GET	<code>/api/patients</code>	Retrieves all registered patients / customers.
POST	<code>/api/patients</code>	Registers a new patient / customer profile.
PUT	<code>/api/patients/:id</code>	Updates an existing patient's profile.
DELETE	<code>/api/patients/:id</code>	Deletes a patient profile (only if no active or past dispense history).
GET	<code>/api/dispense</code>	Lists all currently active (undispensed-return) records

Method	Endpoint Path	Action Description
		with status.
POST	/api/dispense	Creates a new dispense record (medicine handed to patient).
POST	/api/dispense/:id/return	Processes a restock / return and calculates late surcharge.

By organizing the backend into these renamed REST endpoints — while retaining the original route names as legacy aliases under the hood — the server remains completely stateless regarding the client's UI, simply ingesting parameters and returning JSON.

6.3 Legacy Endpoint Compatibility

To guarantee zero breaking changes for any external tooling built against the original schema-era routes, the server also mounts a legacy compatibility layer. The table below shows the mapping between the current pharmacy-domain routes and their retained legacy aliases.

Current Route	Legacy Alias	Notes
/api/medicines	/api/books	Both paths reach the same medicine-inventory router.
/api/patients	/api/members	Both paths reach the same patient-directory router.
/api/dispense	/api/issues	Both paths reach the same dispensing/restock router.
/api/dispense/:id/return	/api/issues/:id/return	Restock/return action, identical behaviour on either path.

7 DATABASE DESIGN

The relational database schema is normalized to prevent data redundancy and ensure logical consistency across pharmacy operations. It consists of three primary tables, which are presented to the user under pharmacy-domain names while the underlying SQL table and column names (shown in parentheses) were left completely unchanged to guarantee backward compatibility.

7.1 Domain Mapping Overview

DB Table	Column	Original Meaning	MediCare Pro UI Label
books	id	Book ID	Medicine ID
books	title	Book Title	Medicine Name
books	author	Author Name	Manufacturer / Brand
books	isbn	ISBN Number	Barcode / Batch Code
books	genre	Book Genre	Drug Category
books	total_copies	Total Copies	Total Stock
books	available_copies	Available Copies	Available Stock
members	id	Member ID	Patient ID
members	name	Member Name	Patient / Customer Name
members	email	Email	Email Address
members	phone	Phone Number	Phone Number
members	membership_date	Date Joined	Registration Date
issues	id	Issue Record ID	Dispense Record ID
issues	book_id	Book ID (FK)	Medicine Product ID
issues	member_id	Member ID (FK)	Patient ID
issues	issue_date	Issue Date	Dispensed Date
issues	due_date	Due Date	Refill / Return Due
issues	return_date	Return Date	Restock Date
issues	fine	Overdue Fine	Late Return Fee / Surcharge (₹5/day)

7.2 Medicines Table (books)

The medicines table stores the inventory catalog. The barcode/batch-code field is marked UNIQUE to prevent cataloguing the exact same product twice under different IDs. The table explicitly separates total stock (units the store has received) from available stock (units currently on the shelf).

Column Name	Data Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
title (Medicine Name)	TEXT	NOT NULL

Column Name	Data Type	Constraints
author (Manufacturer)	TEXT	NOT NULL
isbn (Barcode/Batch Code)	TEXT	NOT NULL UNIQUE
genre (Drug Category)	TEXT	NOT NULL
total_copies (Total Stock)	INTEGER	NOT NULL
available_copies (Available Stock)	INTEGER	NOT NULL

7.3 Patients Table (members)

The patients table stores patient / customer profiles. The email address serves as a unique identifier to avoid registering the same person multiple times.

Column Name	Data Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
name (Patient Name)	TEXT	NOT NULL
email	TEXT	NOT NULL UNIQUE
phone	TEXT	NOT NULL
membership_date (Registration Date)	TEXT	NOT NULL

7.4 Dispense Records Table (issues)

The dispense records table is the transactional heart of the system. It maps the many-to-many relationship between medicines and patients using foreign keys. An active dispense is defined logically as a record where return_date (Restock Date) is NULL.

Column Name	Data Type	Constraints
id	INTEGER	PRIMARY KEY AUTOINCREMENT
book_id (Medicine ID)	INTEGER	NOT NULL, FOREIGN KEY(books.id)
member_id (Patient ID)	INTEGER	NOT NULL, FOREIGN KEY(members.id)
issue_date (Dispensed Date)	TEXT	NOT NULL
due_date (Refill/Return Due)	TEXT	NOT NULL
return_date (Restock Date)	TEXT	NULLABLE
fine (Late Fee)	REAL	DEFAULT 0

8 MODULE DESCRIPTION

The application codebase is divided into modular files to separate concerns. As part of the domain translation, the backend route modules and the database file were renamed to reflect the pharmacy domain; `server.js`, `db.js`, and `generate_report.py` kept their original names since they are domain-neutral entry points. The table below summarises every rename.

8.1 Updated File Names & Project Structure

Original File	New File	Responsibility
<code>library.db</code>	<code>medical.db</code>	SQLite / libSQL relational database file
<code>routes/books.js</code>	<code>routes/medicines.js</code>	Inventory stock API router (<code>/api/medicines</code>)
<code>routes/members.js</code>	<code>routes/patients.js</code>	Patient directory API router (<code>/api/patients</code>)
<code>routes/issues.js</code>	<code>routes/dispense.js</code>	Prescription dispensing & stock-return API router (<code>/api/dispense</code>)
<code>server.js</code>	<code>server.js</code> (unchanged)	Express application server & REST router mounting
<code>db.js</code>	<code>db.js</code> (unchanged)	Database connection module & sample-data seeder
<code>generate_report.py</code>	<code>generate_report.py</code> (unchanged)	Project report generation script

8.2 Backend Route Modules

- `server.js`: The main entry point. It configures the Express application, sets up middleware (CORS, JSON body parser), serves static files from the public directory, mounts the renamed API routers (medicines, patients, dispense) plus their legacy aliases, and defines the aggregate `/api/dashboard` endpoint.
- `db.js`: Handles the libSQL/SQLite database initialization against the renamed `medical.db` file. Its seeding arrays were updated to populate realistic pharmaceutical inventory (sampleBooks) and patient records (sampleMembers) on first run, while the table schemas, constraints, and `db.batch` initialization logic were preserved unchanged.
- `routes/medicines.js` (formerly `routes/books.js`): Contains the logic for validating incoming medicine data (name, manufacturer, drug category, barcode format, and non-negative stock counts) and executing CRUD SQL queries against the books table. It ensures stock math is respected when updating existing medicines, preventing the total from being set below the number currently dispensed.
- `routes/patients.js` (formerly `routes/members.js`): Provides operations for patient records, enforcing unique emails and preventing deletion of patients who have an active or past dispense history.
- `routes/dispense.js` (formerly `routes/issues.js`): The most complex module. It calculates refill/return-due dates fourteen days after dispensing, retrieves active dispense records with SQL JOINS to get readable medicine and patient names, updates stock-availability counts, and calculates late fees upon restock.

- `netlify/functions/api.js`: Wraps the exact same Express app with `serverless-http` so it can run as a Netlify Function, rewriting the incoming function path back to the `/api/...` form the Express routes expect.

8.3 Frontend Modules (Public Directory)

- `index.html`: The single HTML file containing the entire application structure, rebranded as MediCare Pro. It adds Google Fonts (Plus Jakarta Sans and Inter), SVG medical icons (cross, pill, patients, dispense arrows), a repositioned header with a live date chip and quick-operation shortcuts, a branded sidebar with an active-state indicator and store-status dot, four gradient-bordered metric cards on the dashboard, a two-column Prescription Dispense layout, and a mobile sticky top bar plus bottom tab bar. All functional element IDs (`message`, `dashboard`, `books`, `members`, `issues`, `totalBooks`, `totalMembers`, `totalIssued`, `totalOverdue`, `bookSearch`, `addBookButton`, `bookForm`, `booksList`, `memberForm`, `membersList`, `issueFormBox`, `issueForm`, `issueBook`, `issueMember`, `issuesList`) were preserved so the existing `script.js` continues to work unmodified.
- `style.css`: Implements the medical design system — teal/emerald primary colours, medical-cyan accents, a dark-slate sidebar, glassmorphism cards, rounded borders, stock-level badges (`.badge-stock`, including a `.low-stock` warning state), drug-category badges, and status indicators for overdue items — replacing the original navy-and-accent-blue palette while reusing the same responsive grid and media-query breakpoints (`@media max-width: 768px`).
- `script.js`: Contains the browser-side business logic. All UI text, dialog prompts, placeholders, and error messages were updated to reflect medical-store operations (stock pills, category chips, bold medicine names), and a date-chip handler was added for the new header bar — while every `fetch('/api/...')` call, HTTP verb, payload object, and backend error handler was left untouched.

9 WORKFLOW AND DATA INTEGRITY

Ensuring that the pharmacy database remains consistent during concurrent or erroneous operations is critical. The system enforces strict business rules at the API layer, unchanged from the original circulation logic.

9.1 Medicine Dispense Workflow

- 1. Store staff select a medicine and a patient from dropdown lists (which only display medicines with available stock > 0).
- 2. The server receives the POST request and confirms both the medicine and patient IDs exist in the database.
- 3. The server explicitly checks `available_copies` (Available Stock) to ensure it is strictly greater than zero, preventing overdrafts.
- 4. A new dispense record is created using the current system date as the Dispensed Date.
- 5. The server calculates the Refill/Return Due date exactly 14 days in the future.
- 6. Crucially, the dispense-record insertion and the `UPDATE books SET available_copies = available_copies - 1` statement are sent together as one batched write. If one fails, neither change is applied, so the database remains consistent.

9.2 Restock / Return and Late-Fee Workflow

The restock process automates penalty calculations to remove human error:

- 1. Staff click Restock next to an active dispense record on the dashboard.
- 2. The server records the current date as the Restock Date.
- 3. It mathematically compares the restock date against the Refill/Return Due date. If the return is late, the difference in days is multiplied by the surcharge rate (₹5 per day).
- 4. The system updates the dispense record with the restock date and the calculated late fee, and simultaneously increments Available Stock for the medicine by 1, again as a single batched write.

9.3 Input Validation and Deletion Guards

To prevent garbage data, backend routes validate all inputs before SQL execution. Medicines require a name, manufacturer, drug category, a barcode/batch code matching a simple numeric/hyphen pattern (10–17 digits), and a non-negative whole number of stock units. Patients require a name, phone, registration date, and a properly formatted email string.

Furthermore, referential-integrity guards are placed on DELETE routes. The application actively blocks the deletion of any medicine or patient that has an active dispense record, and also blocks deletion if the medicine or patient has any past dispense history at all. Deleting such records would break the relational integrity of the `dispense_records` table, resulting in orphaned records and corrupted dashboard statistics.

10 USER INTERFACE DESIGN

The user interface was redesigned around a clean, clinical-feeling teal-and-cyan medical aesthetic, while preserving the simplicity, speed, and responsiveness of the original Single Page Application (SPA), avoiding the jarring flash of full page reloads.

10.1 Dashboard

The dashboard is the landing view. It presents four prominent metric cards with gradient top borders and medical icons: Total Medicines, Total Patients, Medicines Dispensed, and Overdue Returns. These numbers are fetched via the `/api/dashboard` endpoint and give pharmacy staff an immediate, high-level overview of the store's status the moment they open the application, alongside a grid of quick-operation shortcut tiles.

10.2 Inventory and Patient Views

The Medicines and Patients sections feature prominent action buttons — "+ Add Medicine" and "+ Register Patient" — that open dynamic data-entry forms. The Medicines section includes a real-time text search box (`bookSearch`). As the user types, the JavaScript client queries the backend and filters the HTML table to show only medicines matching the name or manufacturer, allowing for rapid inventory searching.

Data is displayed in clean, striped tables with stock-level badges (including a low-stock warning state) and drug-category chips. Each row features action buttons (Edit and Delete) specific to that record.

10.3 Prescription Dispense Interface

The Dispense section merges functionality into a two-column split layout. The left dispensing card provides a streamlined form with two dropdowns for picking a patient and a medicine. The frontend actively filters the medicine dropdown so that items with zero available stock cannot even be selected by the user.

The right-hand table displays all currently active (undispensed-return) records. A dynamic status column visually flags items as 'On Time' (green) or 'Overdue' (red) based on a real-time date comparison. Clicking 'Restock' immediately processes the transaction and flashes a summary message, including any calculated late fee, at the top of the screen.

10.4 Responsive Mobile Design

Extensive CSS media queries ensure the application is usable on mobile devices. Below the responsive breakpoint, the dark-slate desktop sidebar disappears and is replaced by a sticky mobile top bar and a bottom icon-based navigation bar, similar to modern mobile health apps. Additionally, the standard HTML tables re-flow into stacked 'cards' so that horizontal scrolling is not required to read row data on a smartphone screen.

11 TESTING AND VALIDATION

Due to the compact and educational nature of the project, a manual functional testing strategy was employed. Each API operation was exercised repeatedly through the re-themed UI and cross-referenced with backend database state to ensure absolute logical correctness was preserved after the domain translation.

11.1 Core Test Cases

Test Case	Action / Input	Expected System Outcome
TC01: Add Medicine	Enter valid medicine details with 5 units of stock.	Medicine appears in UI table. Database reflects 5 total, 5 available units.
TC02: Invalid Barcode	Attempt to save a medicine with text in the barcode field.	Server rejects payload (HTTP 400). Error message displayed in UI.
TC03: Dispense Available Medicine	Select a patient and an in-stock medicine.	Dispense record created. Available stock decreases by exactly 1.
TC04: Dispense Out-of-Stock Medicine	Attempt POST request bypassing UI for a 0-stock medicine.	Server strictly rejects request. Database remains unchanged.
TC05: Delete with History	Attempt to delete a patient who has a currently dispensed medicine.	Server blocks deletion to protect referential integrity. Shows error.
TC06: Late Restock	Manually alter database due date to 2 days ago, then process restock in UI.	Available stock increments by 1. System calculates a late fee of ₹10.
TC07: Search Filter	Type 'Paracetamol' in the medicine search box.	Table updates to show only matching medicine records.
TC08: Duplicate Email	Register a patient with an email already in use.	Server rejects with 'Email must be unique.' No duplicate row created.

11.2 Results and Discussion

The completed application successfully passed all manual functional tests, confirming that the domain-translation approach preserved 100% of the original system's correctness. Medicine and patient records can be accurately managed through a clean, pharmacy-branded interface. The critical dispense process automatically generates refill/return-due dates and securely prevents out-of-stock medicines from being dispensed, matching the guarantees of the original circulation system.

The project demonstrates that a highly functional, data-consistent administrative web application can be re-purposed for a new business domain purely at the presentation layer, without touching schema, routes, or business logic, and without relying on massive, bloated frameworks. Plain vanilla JavaScript paired with simple, parameterized SQL queries remains entirely sufficient for the defined scale, and the domain-mapping table in Chapter 7 makes the translation exceptionally easy to trace and audit.

12 CONCLUSION AND FUTURE SCOPE

12.1 Conclusion

MediCare Pro successfully digitizes the fundamental, everyday operations of a small pharmacy or medical store. It maintains highly structured, normalized information about medicines and patients, supports robust dispense and restock activities, automatically calculates late fees, and provides immediate dashboard statistics — all while running on an unmodified, backward-compatible database schema and API.

This project satisfies its primary learning objective of demonstrating a schema-preserving domain-translation methodology: taking a full-stack web application built with HTML, CSS, vanilla JavaScript, Node.js, Express, and a libSQL/SQLite database, and re-purposing it for an entirely different business vertical by re-mapping labels, sample data, and visual theme at the presentation layer only. The implementation intentionally remains beginner-friendly by utilizing clear routing patterns, direct SQL queries, and small, focused JavaScript functions, avoiding unnecessary abstraction. It effectively replaces error-prone manual pharmacy registers with a structured digital workflow, providing a strong foundation for small-store administration.

12.2 Current Limitations

While functional, the system does not yet include user authentication mechanisms (login screens), so anyone with access to the deployed URL can manage medicines, patients, and dispense records. It lacks barcode-scanning hardware integration, which means medicines must be selected manually from dropdowns rather than scanned. There is currently no automated test suite; correctness was verified through the manual test cases described in Chapter 11 rather than a repeatable CI test run. It also does not currently verify prescriptions or restrict controlled substances, which a production pharmacy system would require.

12.3 Future Enhancements

The application's modular architecture — and the domain-translation layer demonstrated here — make it reasonably extensible. Logical future enhancements include:

- **Authentication and Authorization:** Adding a simple login mechanism to create distinct roles for 'Pharmacists' and 'Store Staff'.
- **Hardware Integration:** Adding barcode or QR-code scanning support to instantly populate the dispense/restock forms.
- **Automated Notifications:** Integrating a service like Nodemailer or Twilio to send automated email or SMS refill reminders to patients.
- **Prescription Verification:** Adding an upload/verification step for prescription-only medicines before dispensing.
- **Advanced Reporting:** Adding features to export monthly dispense histories or late-fee collection reports to CSV/Excel formats.
- **Automated Testing:** Adding a small suite of API tests that run automatically on every deployment to Netlify.
- **Further Domain Translations:** Applying the same mapping technique demonstrated in Chapter 7 to additional verticals (e.g., equipment rental, tool-lending libraries) to validate the reusability of the underlying schema.

REFERENCES

1. Node.js Official Documentation. Available at: <https://nodejs.org/docs/>
2. Express.js API Reference and Routing Guide. Available at: <https://expressjs.com/>
3. libSQL / Turso Client Documentation (SQLite-compatible database driver). Available at: <https://docs.turso.tech/>
4. SQLite Official Documentation (query language reference). Available at: <https://www.sqlite.org/docs.html>
5. MDN Web Docs: Fetch API and Vanilla JavaScript DOM Manipulation. Available at: <https://developer.mozilla.org/>
6. Netlify Functions and Redirect Rules Documentation. Available at: <https://docs.netlify.com/>
7. serverless-http Package Documentation. Available at: <https://www.npmjs.com/package/serverless-http>
8. Project Source Code: server.js, db.js, routes/medicines.js, routes/patients.js, routes/dispense.js, netlify/functions/api.js, netlify.toml, generate_report.py, and the public/ frontend (index.html, style.css, script.js).
9. Domain Translation & Architecture Notes: internal project context document describing the Library-to-Pharmacy schema mapping used to build MediCare Pro.
10. Architectural layout and reporting standards adapted from standard B.Tech software engineering guidelines.